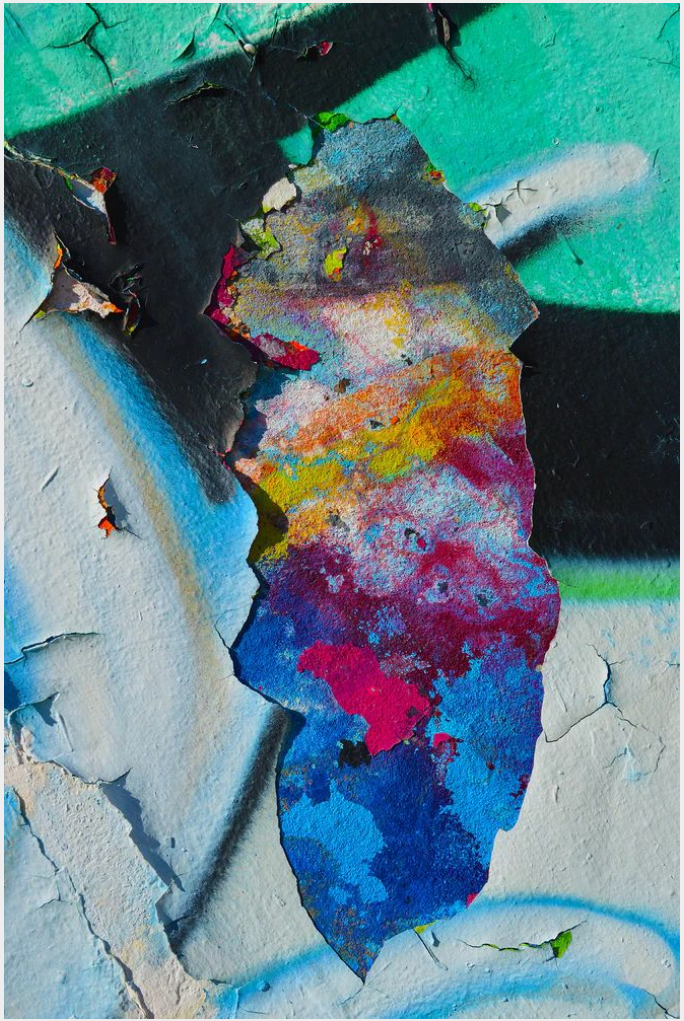


Hubs Cont.

Core Decomposition

Miguel Amaral
miguel.amaral.111@ufrn.edu.br

ivanovitch.silva@ufrn.br
@ivanovitchm

What you already know how to measure...

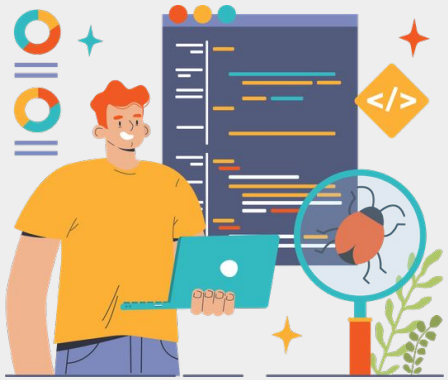


`degree`	how many neighbors do I have?	who is most used
`closeness`	how close am I to everyone?	who reaches others quickly
`betweenness`	how many shortest paths pass through me?	who acts as a bridge
`clustering`	do my neighbors know each other?	how closed is my circle

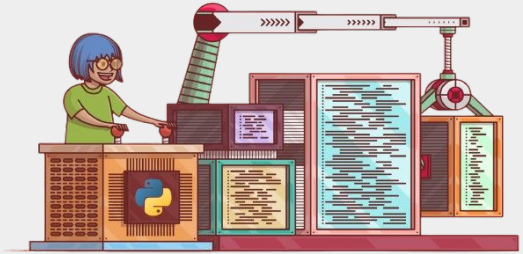


You take over the maintenance of a project ...

Which files can't I touch without breaking everything?



NetworkX
Network Analysis in Python



274 modules.

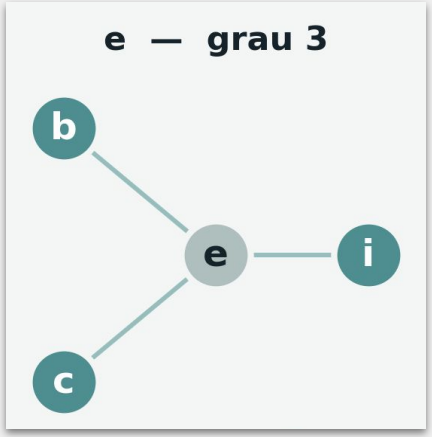
7 modules support the architecture

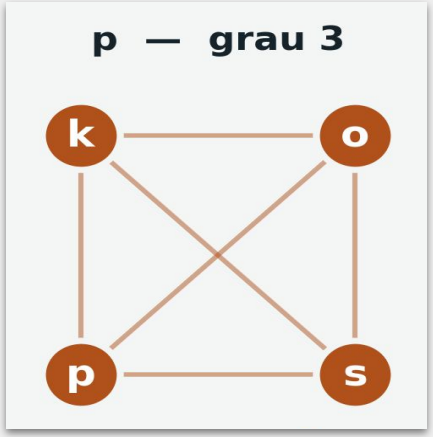
How can we find them?

What about the neighbors?

When it comes to node centrality, one common term you'll hear thrown around is one of **"core" node**.

This is usually a qualitative distinction.



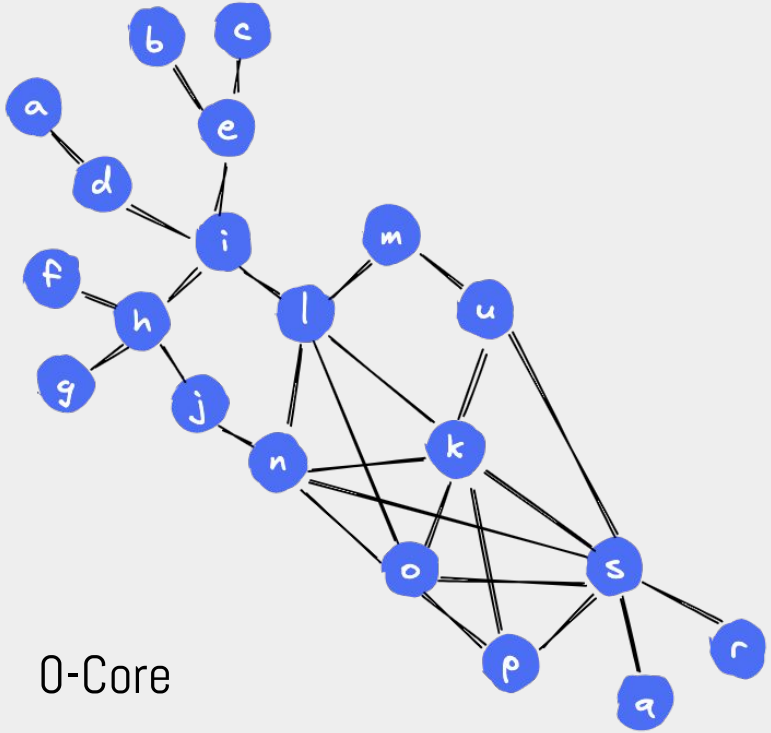


K-Core vs K-Shell

A **k-core** in a network is a subset of its nodes in which all nodes have at least **k connections** to each other. One can easily identify the k-core of a network via the **k-core decomposition algorithm.**



K-Core Decomposition



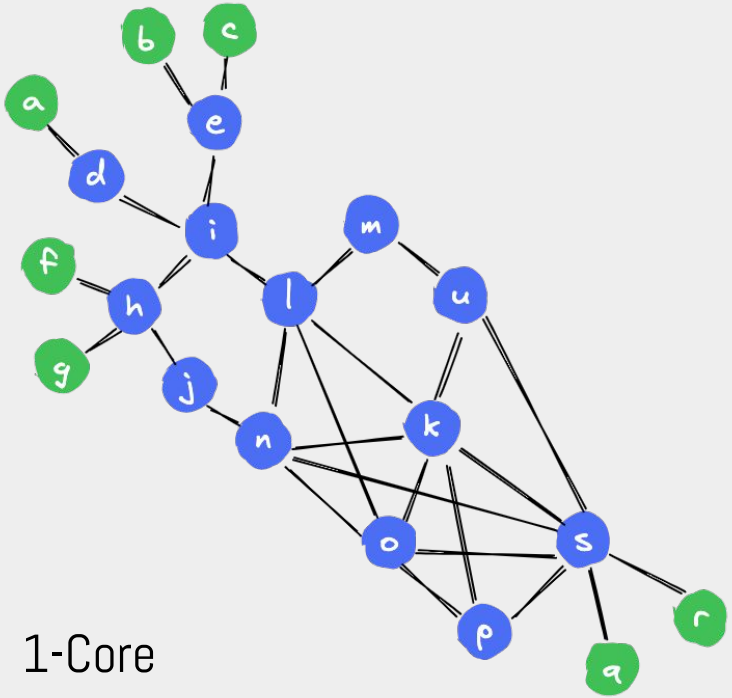
0-Core

```
core = 0
for i in nx.k_core(g,core):
   print(i)
```

```
a
d
b
e
c
i
f
h
g
j
l
n
m
k
o
u
s
p
q
r
```

NetworkX
Network Analysis in Python

K-Core Decomposition



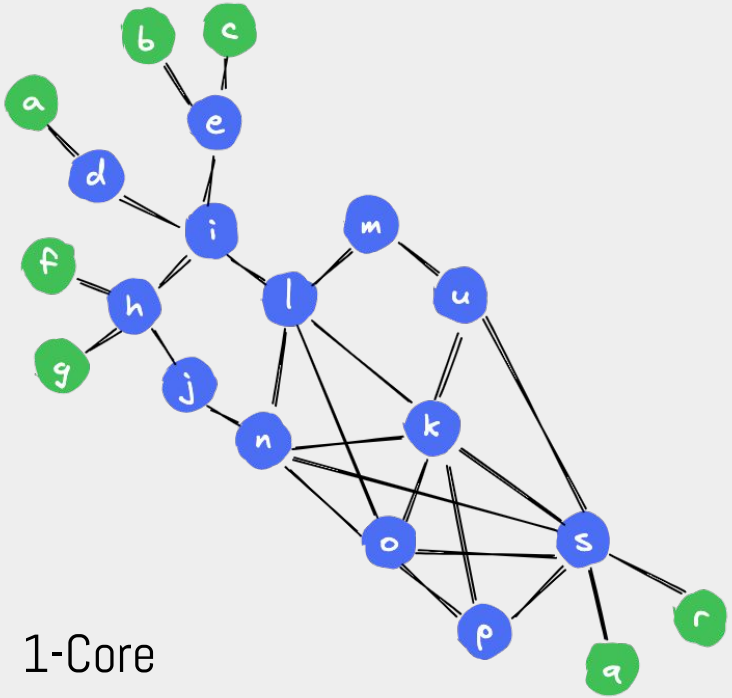
1-Core

```
core = 1
for i in nx.k_core(g,core):
   print(i)
```

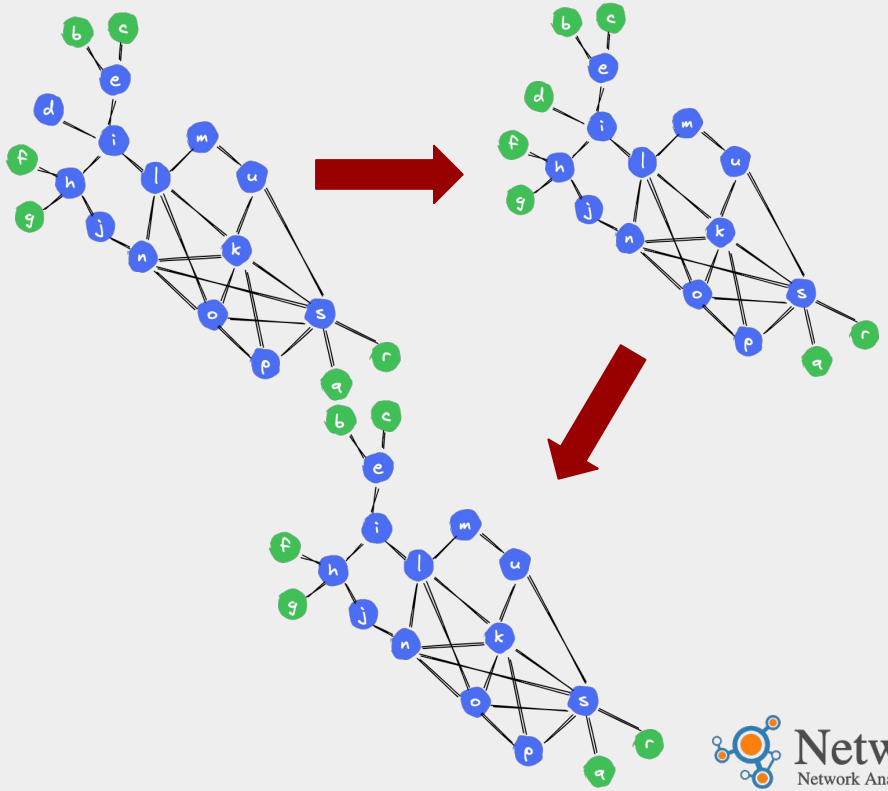
```
a
d
b
e
c
i
f
h
g
j
l
n
m
k
o
u
s
p
q
r
```

NetworkX
Network Analysis in Python

K-Core Decomposition

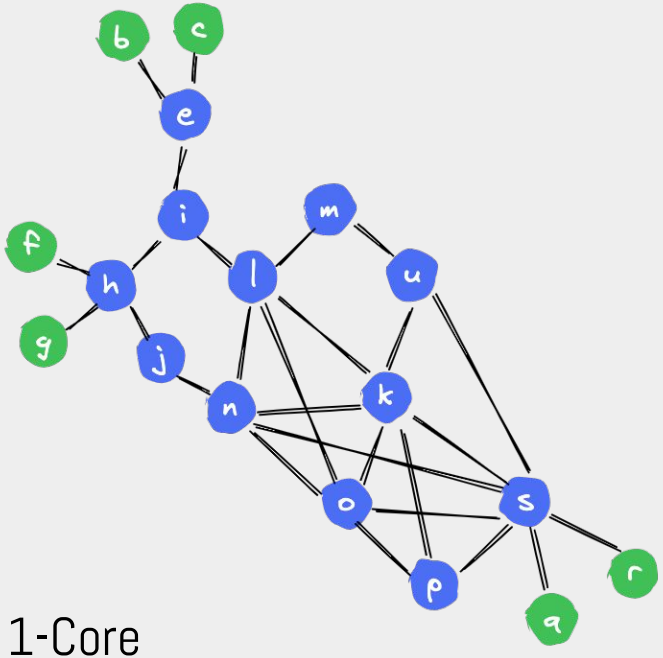


1-Core

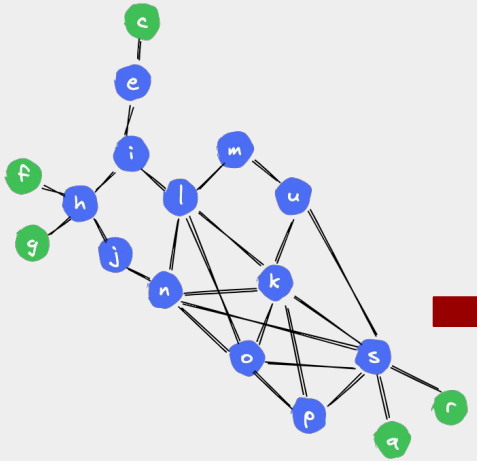


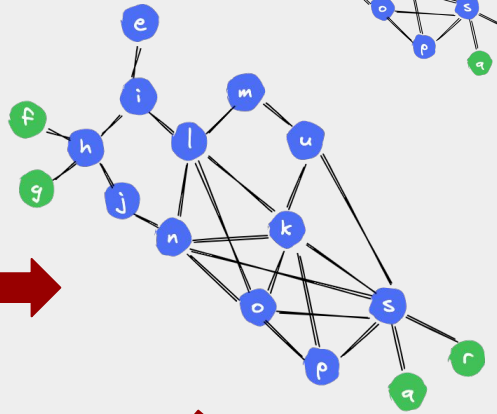
NetworkX
Network Analysis in Python

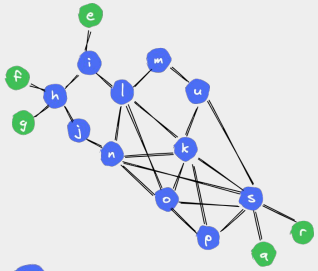
K-Core Decomposition

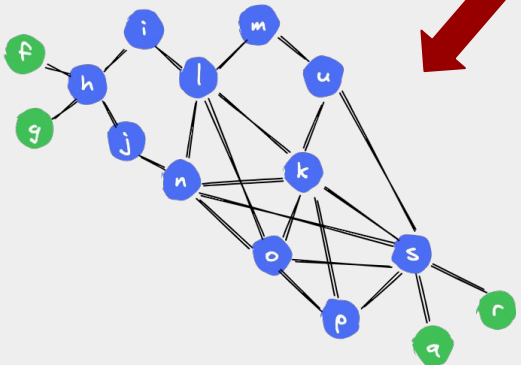


1-Core



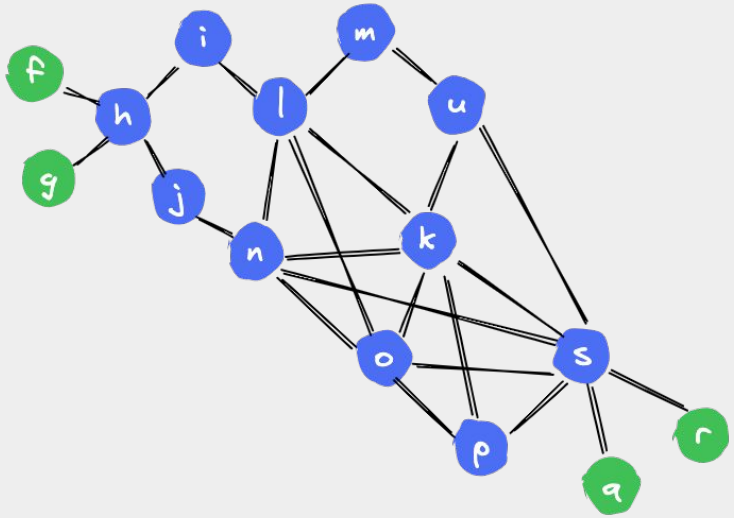




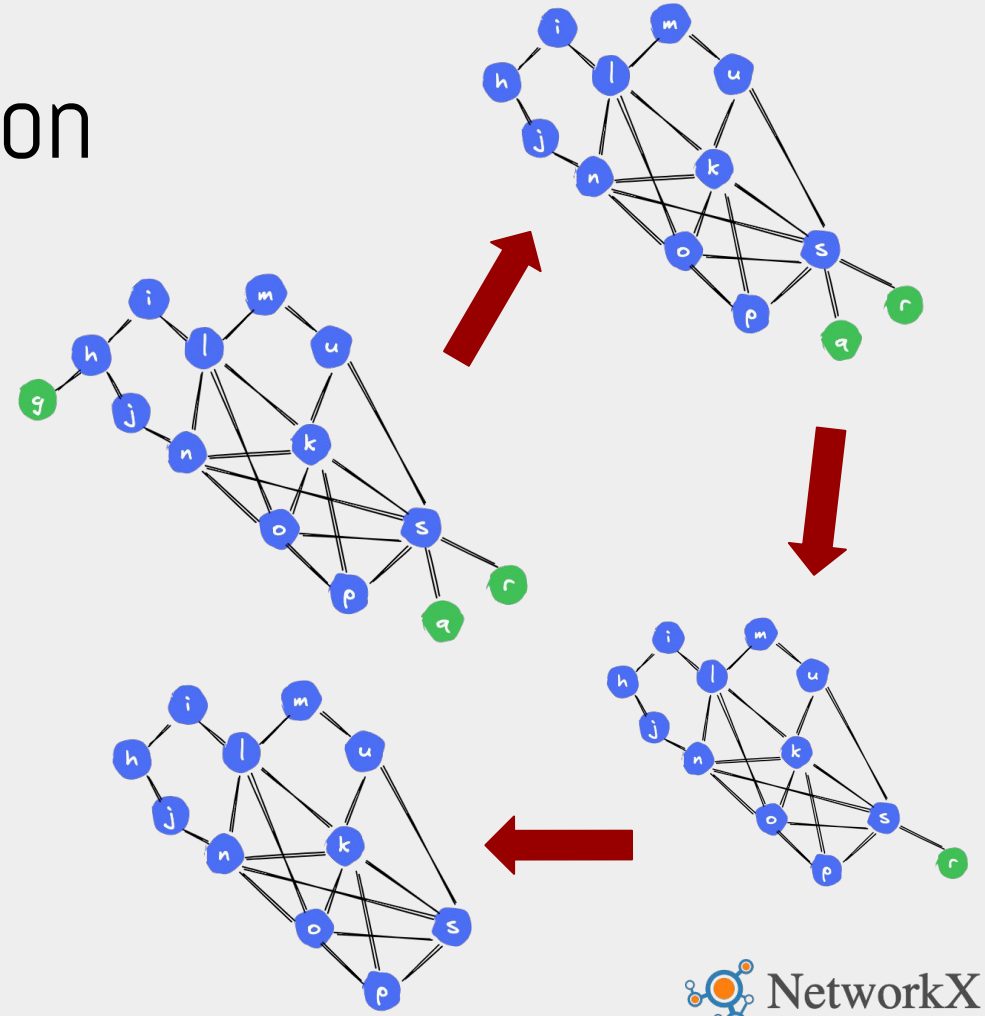


NetworkX
Network Analysis in Python

K-Core Decomposition

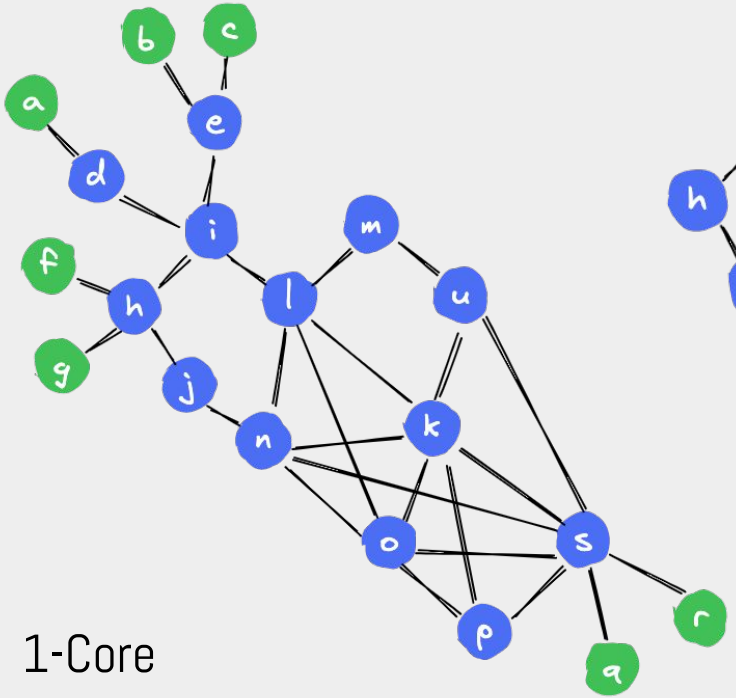


1-Core

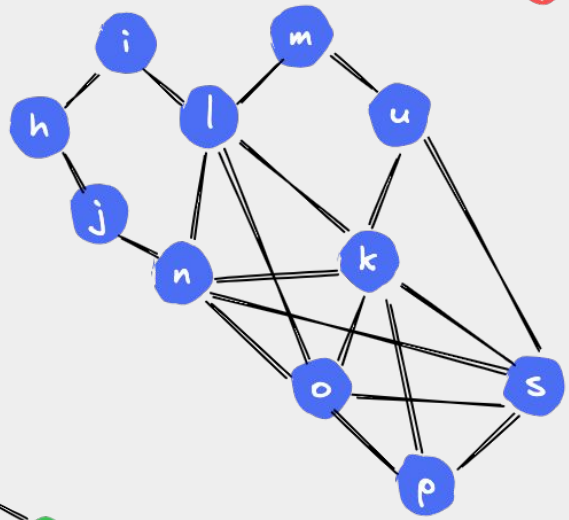


NetworkX
Network Analysis in Python

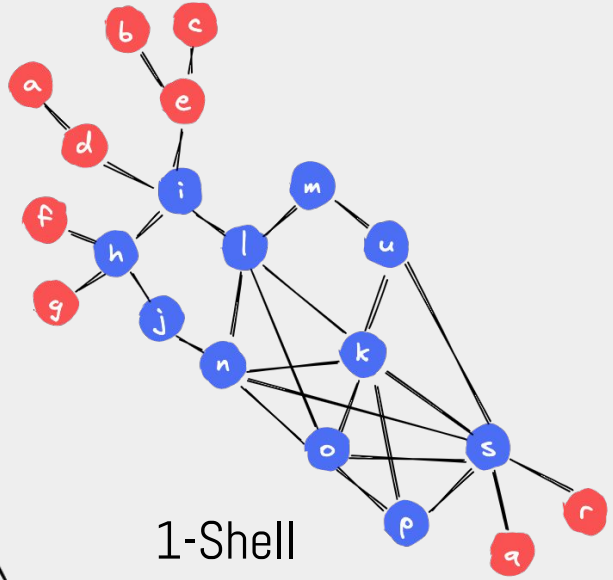
K-Core Decomposition



1-Core



2-Core

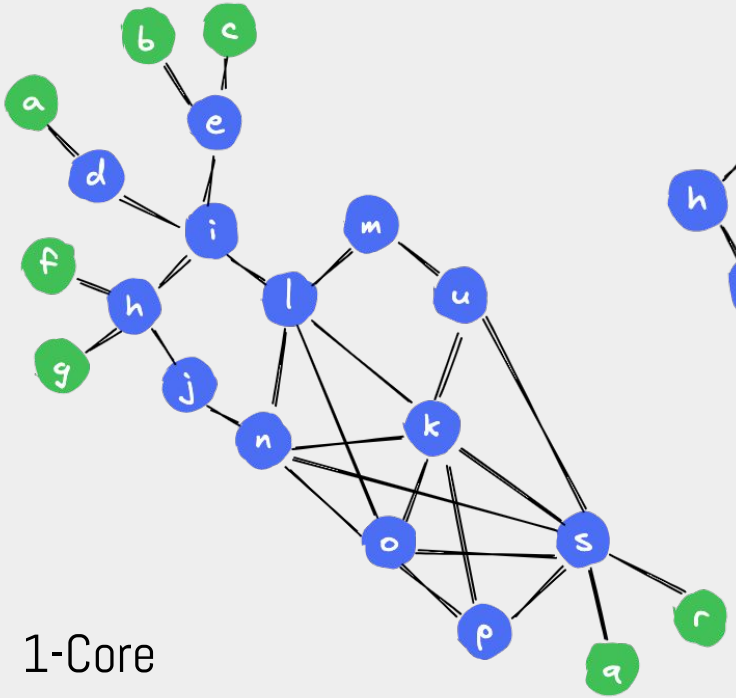


1-Shell

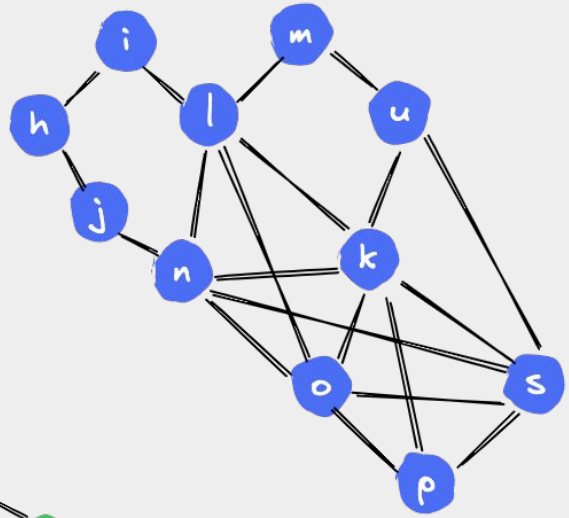


NetworkX
Network Analysis in Python

K-Core Decomposition



1-Core



2-Core

```
core = 2
for i in nx.k_core(g,core):
   print(i)

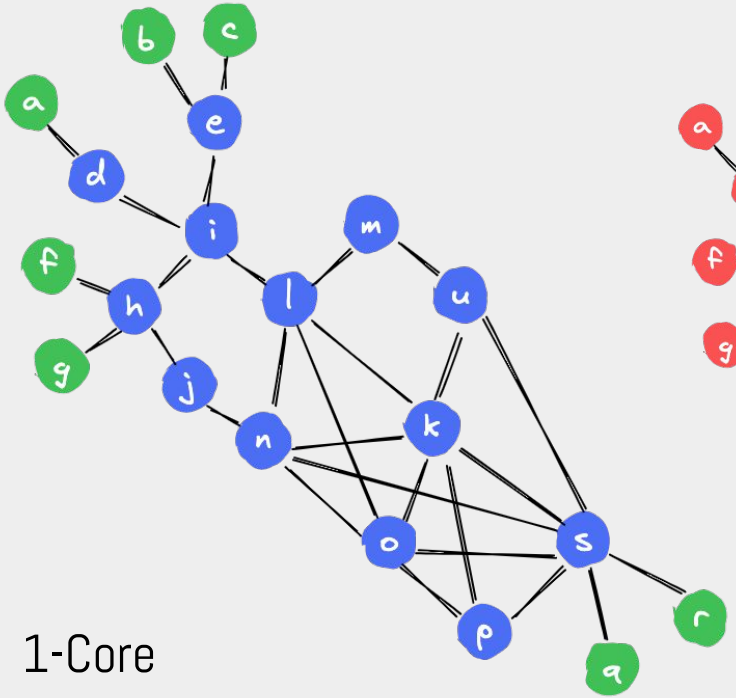
i
h
j
l
n
m
k
o
u
s
p
```

NetworkX
Network Analysis in Python

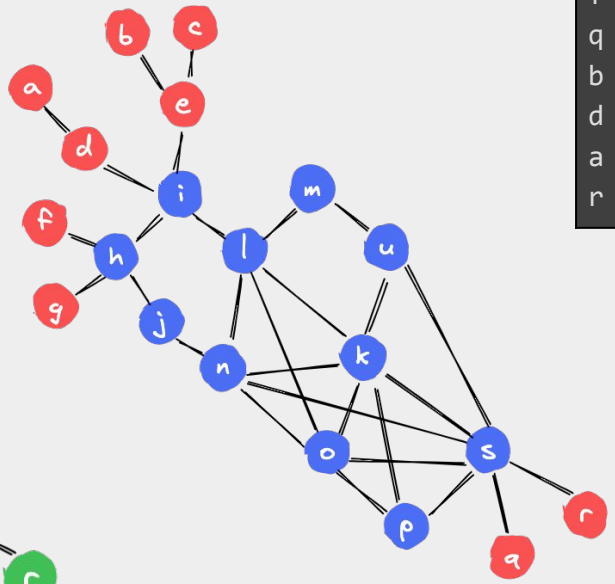
K-Core Decomposition

```
shell = 1
for i in nx.k_shell(g,shell):
 print(i)

g
e
c
f
q
b
d
a
r
```



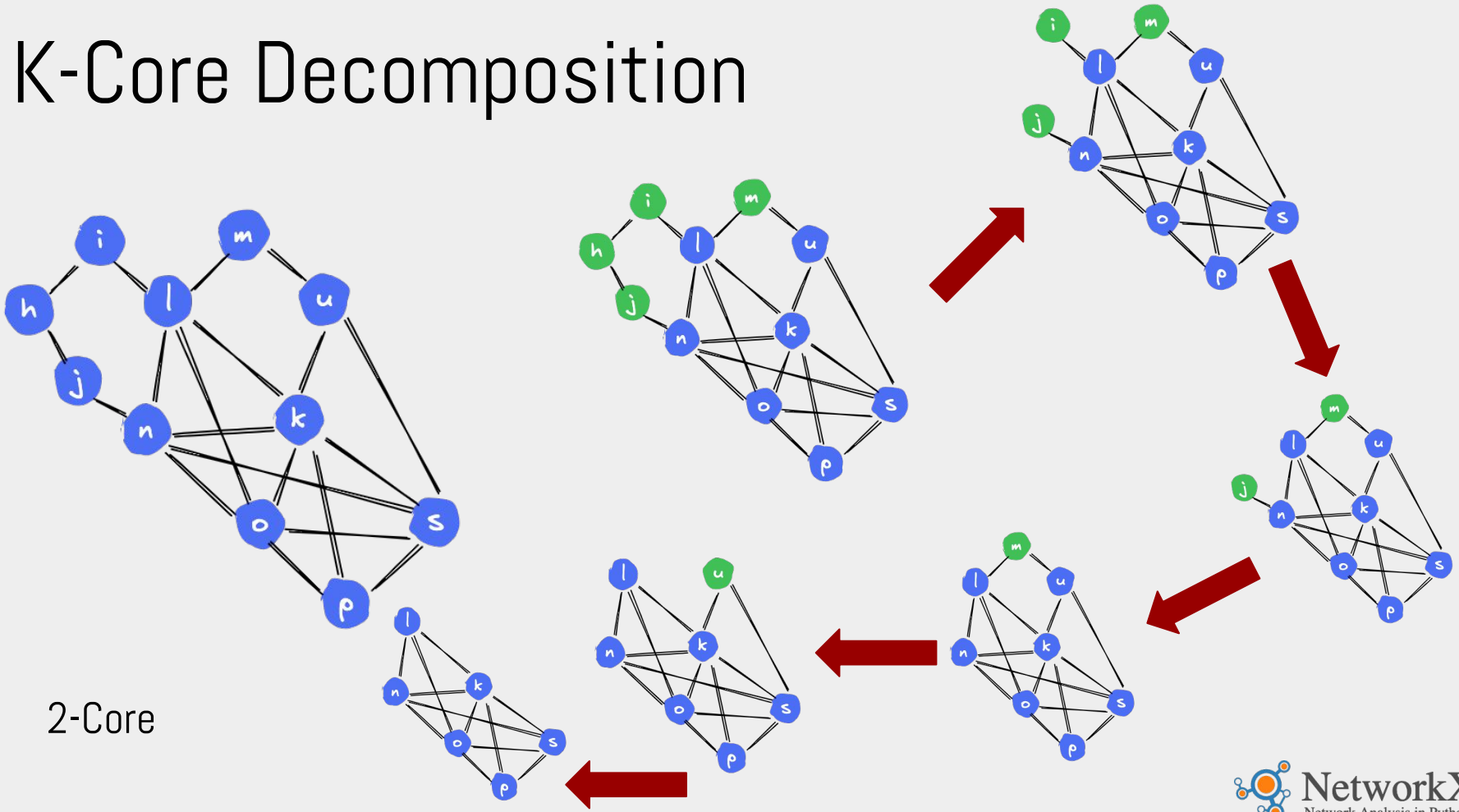
1-Core



1-Shell

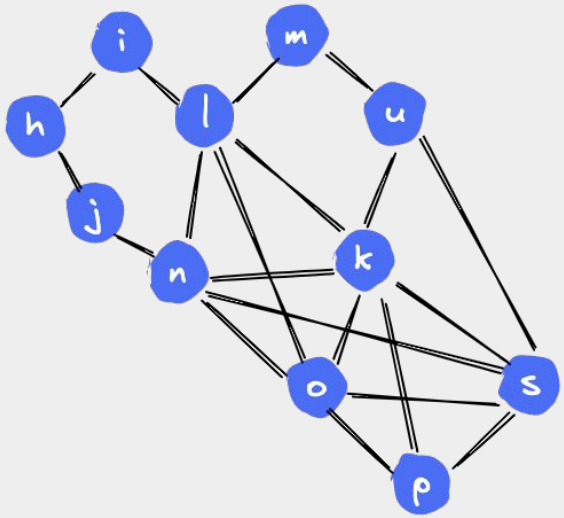
NetworkX
Network Analysis in Python

K-Core Decomposition

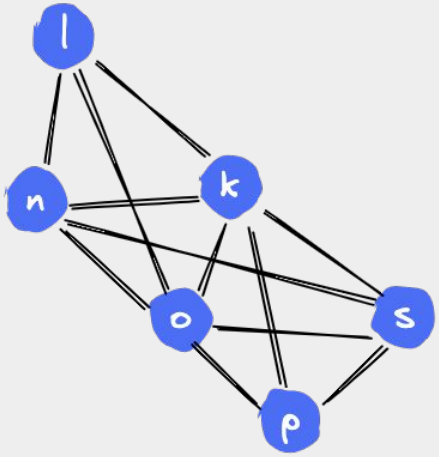


NetworkX
Network Analysis in Python

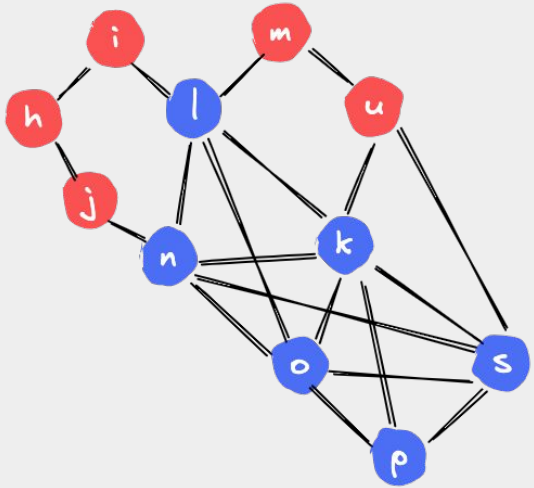
K-Core Decomposition



2-Core



3-Core



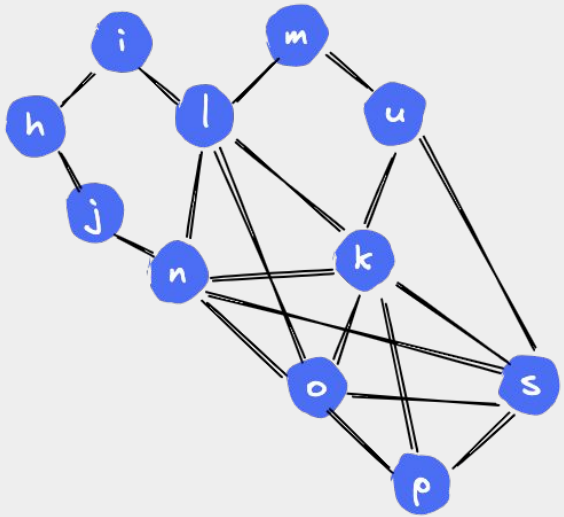
2-Shell

NetworkX
Network Analysis in Python

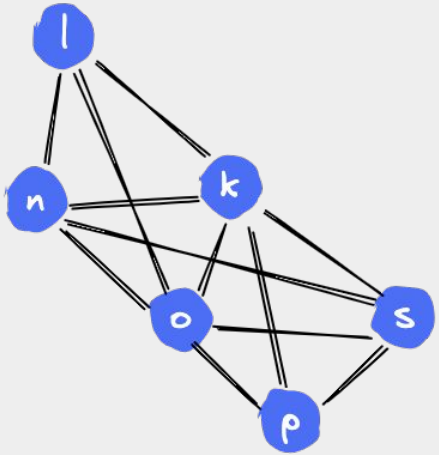
K-Core Decomposition

```
core = 3
for i in nx.k_core(g,core):
   print(i)

o
l
p
k
n
s
```



2-Core

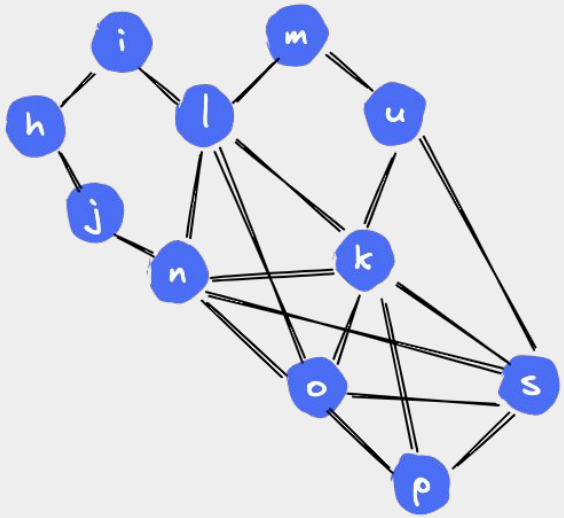


3-Core

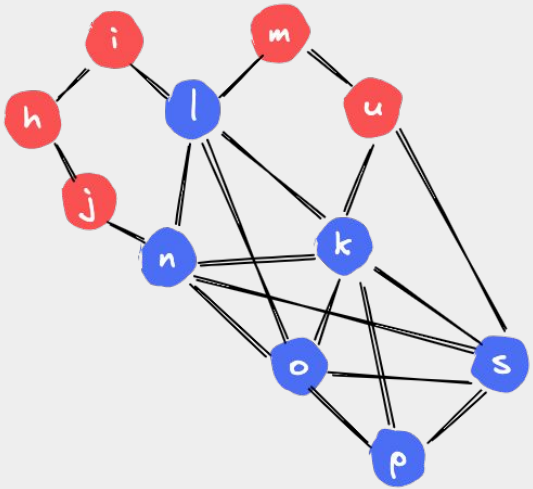
K-Core Decomposition

```
shell = 2
for i in nx.k_shell(g,shell):
 print(i)

u
h
m
j
i
```

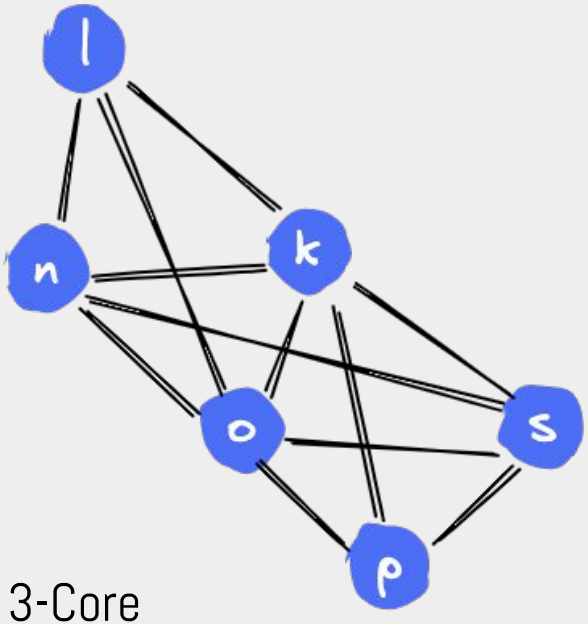


2-Core

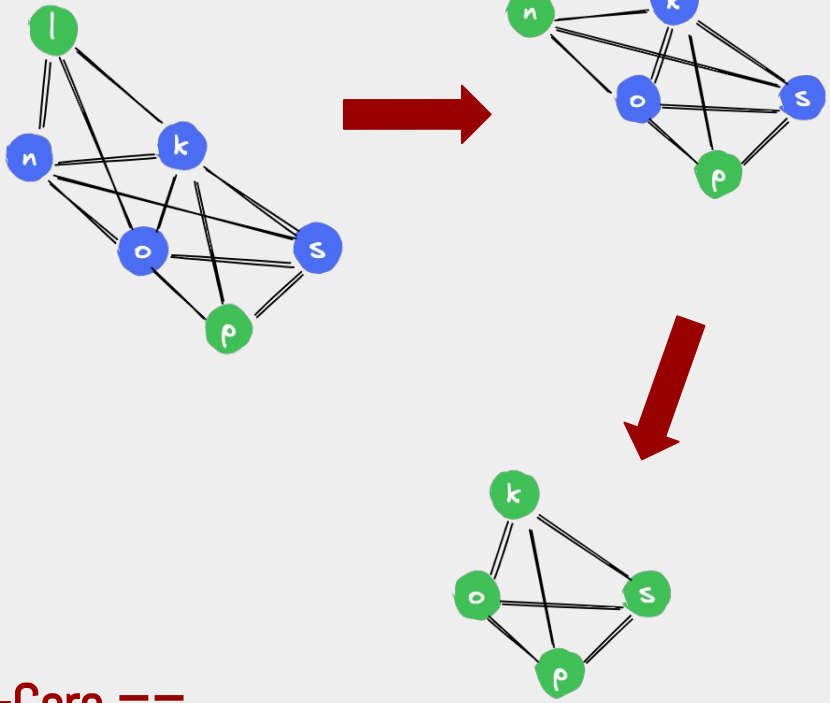


2-Shell

K-Core Decomposition



3-Core



3-Core ==
3-Shell

NetworkX
Network Analysis in Python

K-Core Decomposition

The largest subgraph in which **every** vertex has a degree ≥ k **within that subgraph.**

$$\text{(k+1)-core} \subseteq \text{k-core} \subseteq \ldots \subseteq \text{2-core} \subseteq \text{1-core} \subseteq \text{0-core} = G$$

K-Core Decomposition

coreness(v)

the largest **k** such that **v** belongs to the **k-core**

k-shell

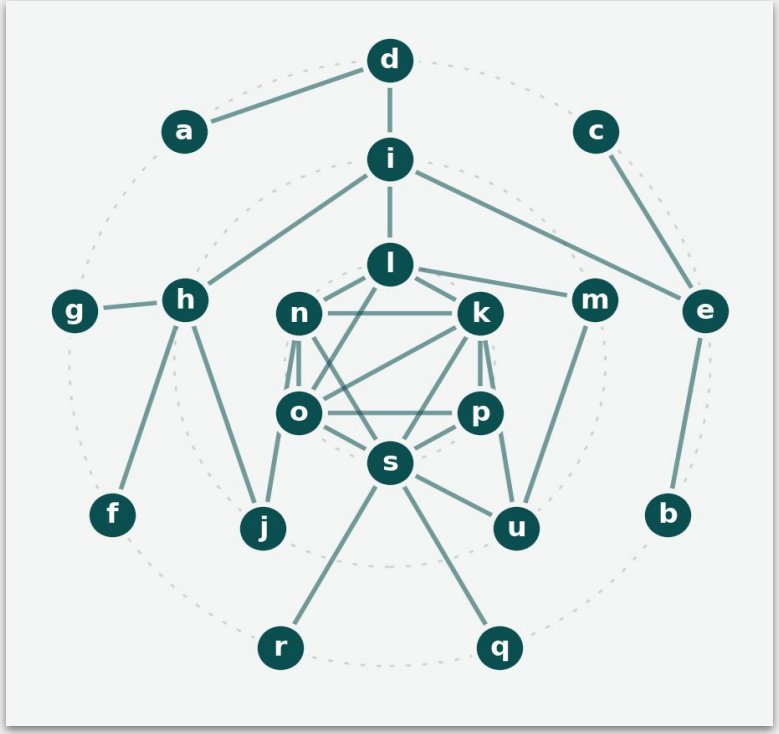
Those with **coreness** exactly k

= k-core − (k+1)-core.

	It's a...	It includes...
k-core	cumulative subgraph	everyone with coreness ≥ k
k-shell	slice — a partition	only those who have coreness exactly k

The k-cores nest. The k-shells partition: each vertex belongs to exactly one.

K-Core Decomposition



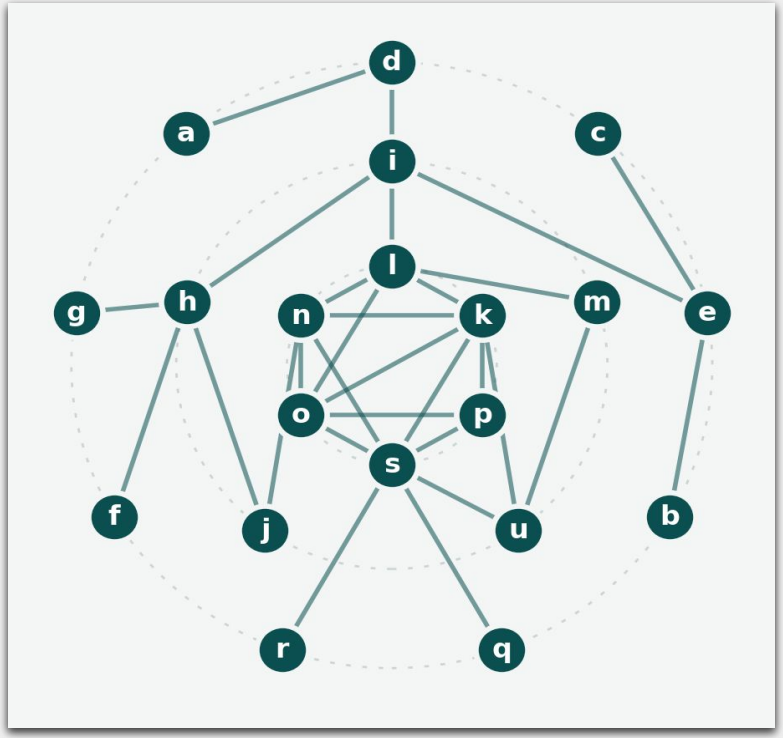
1-core: k = 1

No one leaves. The graph is connected—there are no isolated vertices.

1-core = all 20 vertices

The 0-shell is empty.

K-Core Decomposition



2-core: k = 2

Step 1 Remove a, b, c, f, g, q, r (degree 1)

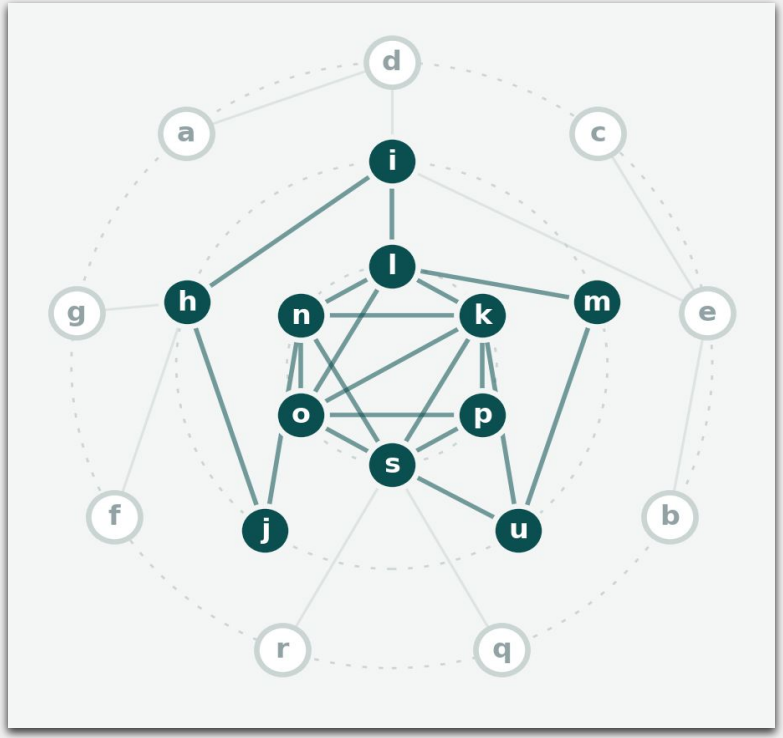
`neighbors drop →  d: 2→1 · e: 3→1 · h: 4→2 · s: 7→5`

Step 2 Remove d, e

2-core = { h,i,j,k,l,m,n,o,p,s,u }

1-shell = {a,b,c,d,e,f,g,q,r}

K-Core Decomposition



3-core: k = 3

Step 1 Remove h, i, j, m (degree 2)

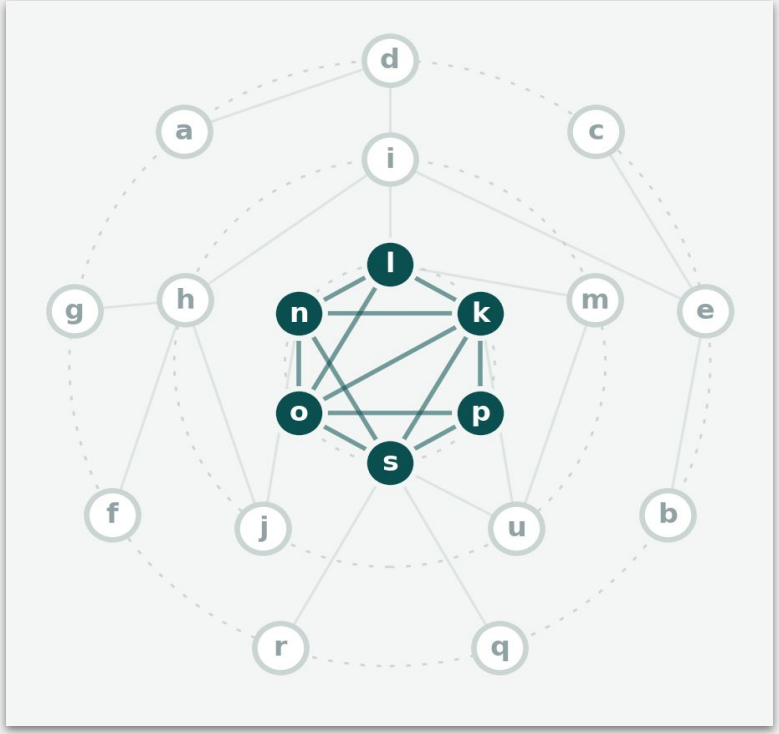
`neighbors drop →  l: 4→3 · n: 5→4 · u: 3→2`

Step 2 Removed u

3-core = { k,l,n,o,p,s }

2-shell = {h,i,j,m,u}

K-Core Decomposition



4-core: k = 4

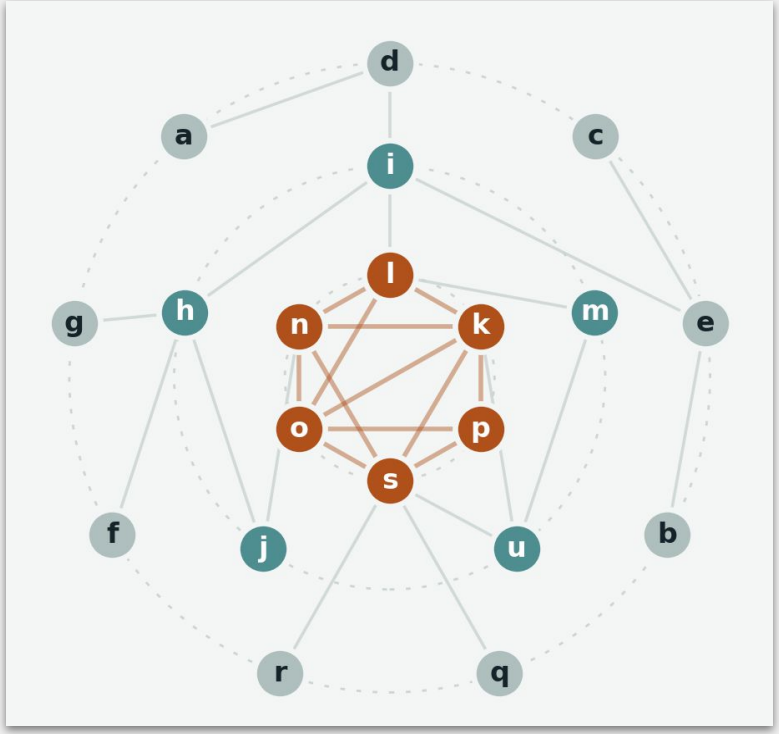
Step 1 Remove l,p (degree 3)

Step 2 k, n, o, and s remain—all with degree 3; they all drop out.

4-core = ∅

Therefore, 3-shell = 3-core, and the graph degeneracy is 3.

K-Core Decomposition



Results

3-core — 6 vertices · minimum degree 3 between each other

k, l, n, o, p, s

2-shell — 5 vertices

h, i, j, m, u

1-shell — 9 vertices — the periphery

a, b, c, d, e, f, g, q, r

https://arxiv.org/pdf/1006.1260.pdf

arXiv:1006.1260v3 [physics.soc-ph] 5 Jan 2011

What's in a crowd?
Analysis of face-to-face behavioral networks

Lorenzo Isella, Ciro Cattuto, and Wouter Van den Broeck
Complex Networks and Systems Group, Institute for Scientific Interchange (ISI) Foundation, Turin, Italy

Juliette Stehlé and Alain Barrat
Centre de Physique Théorique, CNRS UMR 6207, Marseille, France

Jean-François Pinton
Laboratoire de Physique de l'ENS Lyon, CNRS UMR 5672, Lyon, France

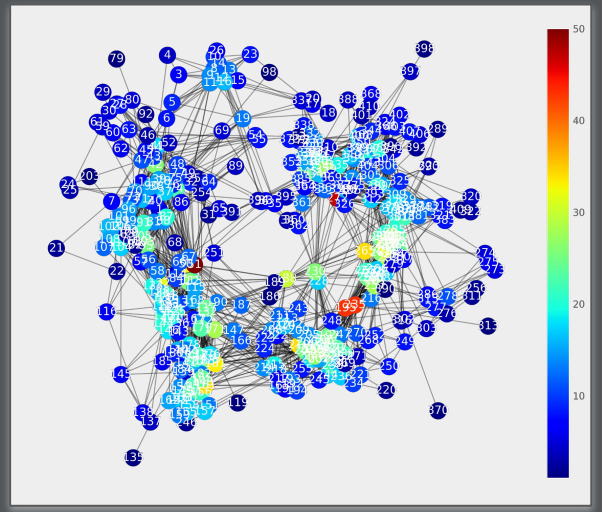
The availability of new data sources on human mobility is opening new avenues for investigating the interplay of social networks, human mobility and dynamical processes such as epidemic spreading. Here we analyze data on the time-resolved face-to-face proximity of individuals in large-scale real-world scenarios. We compare two settings with very different properties, a scientific conference and a long-running museum exhibition. We track the behavioral networks of face-to-face proximity, and characterize them from both a static and a dynamic point of view, exposing differences and similarities. We use our data to investigate the dynamics of a susceptible-infected model for epidemic spreading that unfolds on the dynamical networks of human proximity. The spreading patterns are markedly different for the conference and the museum case, and they are strongly impacted by the causal structure of the network data. A deeper study of the spreading paths shows that the mere knowledge of static aggregated networks would lead to erroneous conclusions about the transmission paths on the dynamical networks.

I. INTRODUCTION

Access to large data sets on human activities and interactions has long been limited by the difficulty and cost of gathering such information. Recently, the ever increasing availability of digital traces of human actions is widely enabling the representation and the analysis of massive amounts of information on human behavior. The representation of this information in terms of complex networks [1–8] has led to many research efforts because of the naturally interlinked nature of these new data sources.

Tracing human behavior in a variety of contexts has become possible at very different spatial and temporal scales: from mobility of individuals inside a city [9] and between cities [10], to mobility and transportation in an entire country [11], all the way to planetary-scale travel [12, 13]. Mobile devices such as cell phones make it possible to investigate mobility patterns and their predictability [14, 15]. On-line interactions occurring between individuals can be monitored by logging instant messaging or email exchange [16–21]. Recent technological advances further support mining real-world interactions by means of mobile devices and wearable sensors, opening up new avenues for gathering data on human and social interactions. Bluetooth and Wifi technologies give access to proximity patterns [22–26], and even face-to-face presence can be resolved with high spatial and temporal resolution [27–30]. The combination of these technological advances and of heterogeneous data sources allow researchers to gather longitudinal data that have been traditionally scarce in social network analysis [31, 32]. A dynamical perspective on interaction networks paves the way to investigating interesting problems such as the interplay of the network dynamics with dynamical processes taking place on these networks.

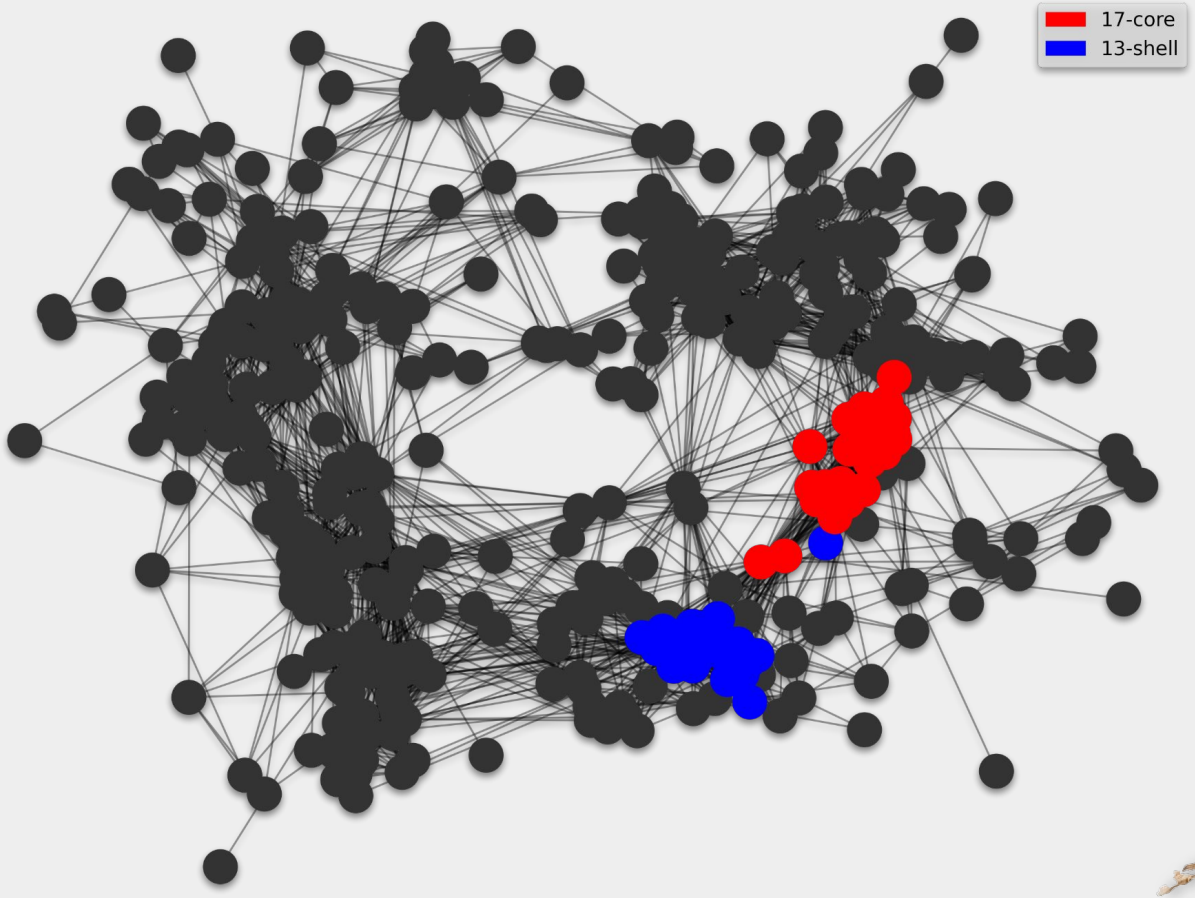
In this paper, we capitalize on recent efforts [27–30] that made possible to mine behavioral networks of face-to-face interactions between individuals, in a variety of real-world settings and in a time-resolved fashion. We present an in-depth analysis of the data we collected at two widely different events. The first event was the INFECTIOUS exhibition [33] held at the Science Gallery in Dublin, Ireland, from April 17th to July 17th, 2009. The second event was the ACM Hypertext 2009 conference [34] hosted by the Institute for Scientific Interchange Foundation in Turin, Italy, from June 29th to July 1st, 2009. In the following, we will refer to these events as SG and HT09, respectively. Intuitively, interactions among conference participants differ from interactions among museum visitors, and the concerned individuals have very different goals in both settings. The study of the corresponding networks of proximity and interactions, both static and dynamic, reveals indeed strong differences but also interesting similarities. We take advantage of the availability of time-resolved data to show how dynamical processes that can unfold on the close proximity network — such as the propagation of a piece of information or the spreading of an infectious agent — unfold in very different ways in the investigated settings. In the epidemiological literature, traditionally,



How many k-core does this network have?

```
# how many k-cores does this network have?
set([v for k,v in nx.core_number(g).items()])
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 17}
```

```
# who are in the innermost k-core
list(nx.k_shell(g,17))
[257, 258, 259, 260, 261, 262, 263, 264, 265,
266, 267, 281, 282, 283, 284, 285, 287, 288, 291,
292, 293, 294, 295, 296, 297, 298, 299, 300, 301,
314, 195, 235]
```



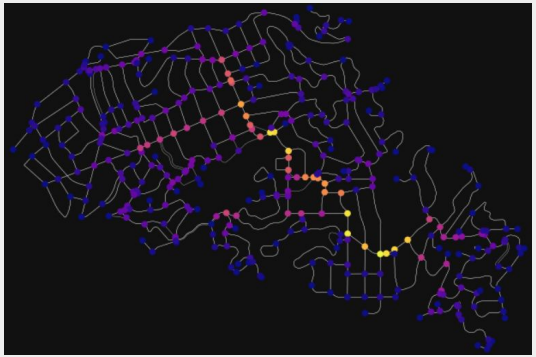

```
# how many k-cores does this network have?
set([v for k,v in nx.core_number(g).items()])
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 17}
```

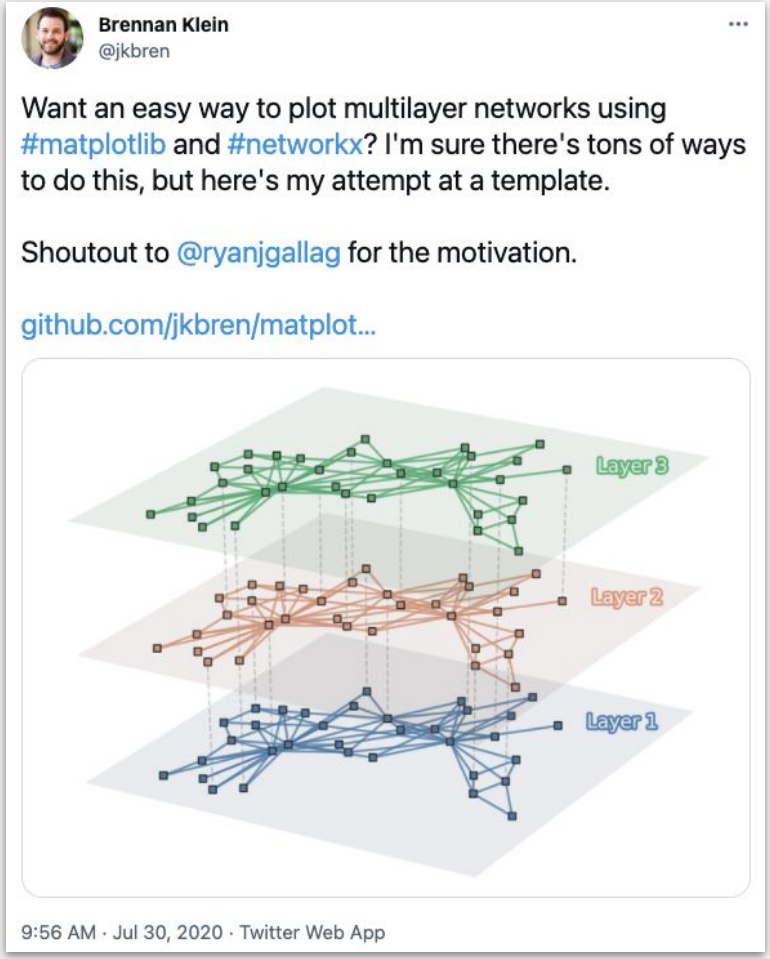


NetworkX
Network Analysis in Python

Further Analysis

Investigate the PDF, CDF, univariate and bivariate analysis when considering the centrality measurements for each **k-shell** and maximum **k-core** of network.





https://twitter.com/jkbren/status/1288820743963324421